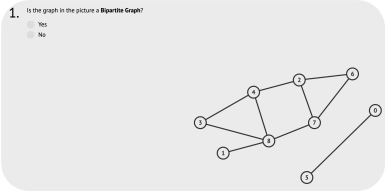


How to tell if graph is bipartite

No.

That graph is **not bipartite** because it contains an **odd cycle**:
3 → 4 → 8 → 3

That is a triangle, which has length 3. Any graph with an odd cycle cannot be bipartite.

How to tell in general:

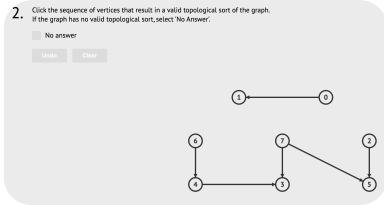
A graph is bipartite if you can split its vertices into **two groups** so that every edge goes between the groups, never within the same group.

Two easy tests:

1. **Look for odd cycles**
   If there is any cycle of length 3, 5, 7, etc., the graph is **not** bipartite.
2. **Try 2-coloring**
   Color one vertex red, all its neighbors blue, their neighbors red, and so on.
   If you ever need to give the same vertex both colors, the graph is **not** bipartite.

For your picture, the triangle $3, 4, 8$ already proves the answer is **No**.

If you want, I can also show the 2-coloring attempt on this exact graph.



What about this

This one **does** have a valid topological sort.

The directed edges are:

- $0 \to 1$
- $6 \to 4$
- $4 \to 3$
- $7 \to 3$
- $7 \to 5$
- $2 \to 5$

There is **no directed cycle**, so a topological ordering exists.

One valid order is:

0, 2, 6, 7, 1, 4, 5, 3

Why it works:

- 0 comes before 1
- 6 comes before 4, and 4 comes before 3
- 7 comes before 3 and 5
- 2 comes before 5

So for this question, **do not choose "No answer."** Select a valid sequence such as:

0 → 2 → 6 → 7 → 1 → 4 → 5 → 3

There are other correct orders too.

How to do this type of question

These problems follow a very standard pattern. Once you know the method, they become quick.

🧠 How to solve topological sort questions

Step 1: Check if an answer is even possible

A topological sort exists **only if the graph has no directed cycles (DAG)**.

Quick ways to check:

- Look for a loop like:
  $A \to B \to C \to A$ → ❌ No answer
- If you suspect a cycle → choose **"No Answer"**

Step 2: Find nodes with no incoming edges (in-degree = 0)

These nodes can go **first** because nothing depends on them.

From your graph:

- $0, 2, 6, 7$ have no incoming edges

Step 3: Remove one and update

Pick any node with in-degree 0:

- Write it down
- Remove it and its outgoing edges
- Update the graph

Example:

- Remove 0 → now 1 becomes available

Step 4: Repeat

Keep doing:

- Find nodes with no incoming edges
- Add to ordering
- Remove them

Step 5: Continue until:

- ✅ All nodes used → valid topological order
- ❌ You get stuck (nodes remain but none have in-degree 0) → cycle → **No answer**

🔁 Example (your graph idea)

Start:

- Available: **0, 2, 6, 7**

Pick in any order (this is why multiple answers exist)

One run:

1. Pick 0 → unlocks 1
2. Pick 2 → helps unlock 5 later
3. Pick 6 → unlocks 4
4. Pick 7 → helps unlock 3 & 5
5. Pick 1
6. Pick 4 → unlocks 3
7. Pick 5
8. Pick 3

⚡ Key Tips (for exams)

- Always start with **nodes with no arrows pointing in**
- Multiple choices = multiple valid answers
- If stuck early → likely a **cycle**
- Think: *"What can I do first without violating dependencies?"*

💡 Shortcut mindset

Instead of memorizing:

> Think of it like tasks with prerequisites

- You can only do a task if all its prerequisites are done
- Build the order step by step

If you want, I can walk you through another example step-by-step or show a faster trick for spotting cycles instantly.

DFS and BFS can run in O(V^2) on a:
Directed Acyclic Graph
Tree
Bipartite Graph
Complete Graph

The correct answer is: **Complete Graph**

🧠 Why?

The time complexity of **DFS** and **BFS** depends on how the graph is represented:

- With an **adjacency list** → $O(V + E)$
- With an **adjacency matrix** → $O(V^2)$

🔍 Key idea

A **complete graph** has the **maximum number of edges**:

- Number of edges:

$$E = \frac{V(V-1)}{2} \approx O(V^2)$$

So even with an adjacency list:

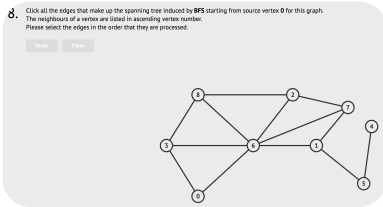
- $O(V + E) = O(V + V^2) = O(V^2)$

📊 Compare options

- **Directed Acyclic Graph (DAG)** → could be sparse → not necessarily $O(V^2)$
- **Tree** → $E = V - 1$ → $O(V)$
- **Bipartite Graph** → can be sparse or dense → not guaranteed $O(V^2)$
- **Complete Graph** → always dense → $O(V^2)$

✅ Final Answer:

Complete Graph



What about this

For the BFS spanning tree starting at **0**, with neighbors considered in **ascending order**, click these edges in this order:

0–3, 0–6, 3–8, 6–1, 6–2, 6–7, 1–5, 5–4

How to get it:

Start at **0**
Neighbors of 0: **3, 6**
Tree edges: **0–3, 0–6**

Queue: [3, 6]

Process **3**
Neighbors: **0, 6, 8**

New vertex: **8**
Tree edge: **3–8**

Queue: [6, 8]

Process **6**
Neighbors: **0, 1, 2, 3, 7, 8**
New vertices in order: **1, 2, 7**
Tree edges: **6–1, 6–2, 6–7**

Queue: [8, 1, 2, 7]

Process **8**
Neighbors already visited

Process **1**
Neighbors: **5, 6, 7**
New vertex: **5**
Tree edge: **1–5**

Queue: [2, 7, 5]

Process **2**
No new vertex

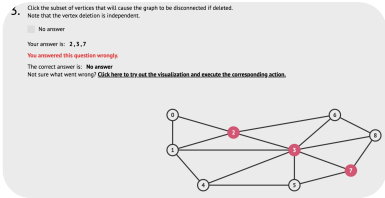
Process **7**
No new vertex

Process **5**
Neighbors: **1, 4**
New vertex: **4**
Tree edge: **5–4**

So the final BFS tree edge order is:

0–3 → 0–6 → 3–8 → 6–1 → 6–2 → 6–7 → 1–5 → 5–4

A quick rule for these questions: in BFS, when you visit a node, scan its neighbors in the given order and add an edge only when that neighbor is first discovered.



Explain this

The key phrase in this question is:

> **"vertex deletion is independent"**

This is what caused the confusion.

🧠 What the question is *actually* asking

You are looking for vertices that, **when removed one at a time (individually)**, make the graph disconnected.

👉 These are called **articulation points (cut vertices)**

❌ Why your answer (2, 3, 7) was wrong

You picked a **set of vertices together** that disconnects the graph.

But the question wants:

> Vertices that **individually** break the graph

So we must test **each vertex separately**.

🔍 Check each of your choices

Vertex 2

- Remove 2
- The graph is still connected via:
  - 0–1–3–6–8 etc.
    ✅ Still connected → NOT valid

Vertex 3

- Remove 3
- Looks important, but:
  - Left side still connects through 1–2–6–8
  - Bottom connects via 4–5–7–8
    ✅ Still connected → NOT valid

Vertex 7

- Remove 7
- Node 8 still connects through 6 and 3
  ✅ Still connected → NOT valid

💡 Big insight

This graph has **lots of alternate paths (cycles)**

So removing any single vertex does **not** break connectivity.

👉 That means:

✅ There are NO articulation points

✅ Final Answer:

No answer

🧠 How to solve these fast (exam trick)

For each vertex:

Ask:

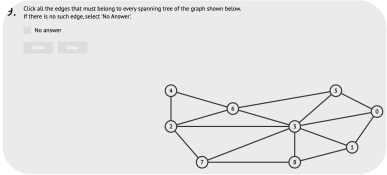
> "If I delete this node, is there still another way around?"

- If **yes** → not an articulation point
- If **no** → it is one

⚡ Shortcut intuition

- Graphs with many triangles / loops → usually **no articulation points**
- Tree-like graphs → **many articulation points**

If you want, I can show you a quick DFS-based trick to find articulation points in seconds (super useful for exams).



How to do this

This question is asking for the edges that are in **every** spanning tree.

Those edges are exactly the **bridges** of the graph.

A **bridge** is an edge whose removal disconnects the graph.

So the method is:

1. Pick an edge.
2. Imagine deleting it.
3. Ask: **Can the two ends still reach each other some other way?**

- If **no**, it is a **bridge** → must be in every spanning tree.
- If **yes**, it is **not** required.

For this graph, every edge lies on a cycle, so every edge has an alternate route.

Examples:

- Edge **4–2** is not required because you can still go **4–6–2**.
- Edge **6–5** is not required because you can still go **6–3–5**.
- Edge **3–8** is not required because you can still go **3–1–8**.
- Edge **0–1** is not required because you can still go **0–3–1**.

Since every edge has another path around it, there are **no bridges**.

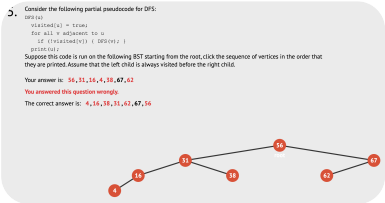
So the correct choice is:

No answer

Quick exam shortcut:

- **Edges in every spanning tree = bridges**
- **An edge on a cycle is never a bridge**

Here, all edges are on cycles, so none must appear in every spanning tree.



Explain this

The important line is:

```
print(u);
```

It happens **after** visiting all children.

So this DFS prints nodes in **postorder**, not preorder.

For this tree:

- Root: **56**
- Left subtree of 56 is rooted at **31**
  - Left child of 31 is **16**
    - Left child of 16 is **4**
  - Right child of 31 is **38**
- Right subtree of 56 is rooted at **67**
  - Left child of 67 is **62**

Since left child is visited before right child, the traversal is:

1. Go from **56 → 31 → 16 → 4**
2. **4** has no children, so print **4**
3. Back to **16**, done with children, print **16**
4. Back to **31**, now visit right child **38**
5. **38** has no children, so print **38**
6. Back to **31**, done, print **31**
7. Back to **56**, now visit right subtree: **67 → 62**
8. **62** has no children, so print **62**
9. Back to **67**, done, print **67**
10. Back to **56**, done, print **56**

So the printed order is:

4, 16, 38, 31, 62, 67, 56

Why your answer was wrong:
You wrote the order more like a top-down visit. But this code does **not** print when a node is first seen. It prints only when recursion returns from that node.

A quick memory trick:

- **print before recursive calls** → preorder
- **print between left and right** → inorder
- **print after recursive calls** → postorder

This question is **postorder DFS**.

What if it is BFS

Good question—this is where people often mix things up.

🧠 If it were BFS instead of DFS

BFS does **level-order traversal** (top → down, left → right).

So for your tree:

```
        56
      /    \
    31      67
   /  \     /
 16   38   62
 /
4
```

🚶 BFS process (using a queue)

Start at root:

Step-by-step:

- Visit **56**
- Then its children: **31, 67**
- Then next level: **16, 38, 62**
- Then last level: **4**

✅ BFS order:

56, 31, 67, 16, 38, 62, 4

🔄 Key difference (super important)

Traversal	Order idea	For this tree
DFS (postorder)	bottom-up	4,16,38,31,62,67,56
BFS	level-by-level	56,31,67,16,38,62,4

⚡ Quick exam trick

- **BFS → think "layers"**
- **DFS → think "go deep first"**

💡 One-liner memory

- BFS = **queue + levels**
- DFS = **recursion + depth**

If you want, I can show how BFS would look if the print was placed differently (like in code form).

How many different spanning trees are there in a complete graph with n vertices?

The number of different spanning trees in a complete graph with n vertices is given by **Cayley's Formula**:

$$\boxed{n^{n-2}}$$

🧠 What this means

For a **complete graph** (every pair of vertices is connected), the number of spanning trees grows very fast:

Examples:

- $n = 1$: $1^{-1} = 1$ (by convention)
- $n = 2$: $2^0 = 1$
- $n = 3$: $3^1 = 3$
- $n = 4$: $4^2 = 16$
- $n = 5$: $5^3 = 125$

💡 Intuition (quick idea)

- A spanning tree connects all n vertices with exactly $n - 1$ edges
- In a complete graph, you have **maximum flexibility** to choose edges
- Cayley's formula counts all possible ways to connect them without cycles

---

✅ Final Answer:

$$\boxed{n^{n-2}}$$

If you want, I can show you a super quick way to derive or remember this for exams.

How to 'Draw a simple connected graph with n vertices and n edges such that the graph contains m bridges.'

Use this construction:

Start with a **cycle** on $n - m$ vertices, then attach the remaining m vertices as a **path** to one vertex of the cycle.

Why this works:

- A cycle with $n - m$ vertices has

$$n - m \text{ vertices and } n - m \text{ edges}$$

- A path of m new vertices adds

$$m \text{ vertices and } m \text{ edges}$$

- Total becomes

$$(n - m) + m = n \text{ vertices}, \quad (n - m) + m = n \text{ edges}$$

So the graph has exactly n vertices and n edges.

Now count bridges:

- In the cycle, **no edge is a bridge**
- In the attached path, **every edge is a bridge**

So the graph has exactly m bridges.

A picture idea:

- Draw a cycle C_{n-m}
- Pick one cycle vertex
- From that vertex, draw a chain of m extra vertices

Example for $n = 8,\ m = 3$:

- Draw a cycle on 5 vertices
- Attach a path of length 3 to one cycle vertex

That gives:

- 8 vertices
- 8 edges
- exactly 3 bridges

Condition:
This works when $0 \le m \le n - 3$, because you need at least a 3-cycle to keep the graph simple and connected.

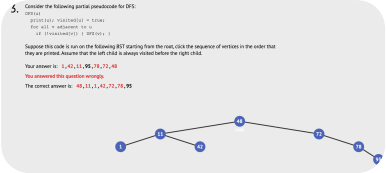
Examples of edge cases:

- $m = 0$: just draw a cycle on all n vertices
- $m = n - 3$: draw a triangle and attach a path of length $n - 3$

So the standard answer is:

$$\boxed{\text{Take } C_{n-m} \text{ and attach a path of length } m \text{ at one vertex.}}$$

If you want, I can also give you a formal proof or a small labeled diagram.



This is inorder?

No — this is **NOT inorder**.

🧠 What traversal is this?

Look at the pseudocode:

```
print(u);
visited[u] = true;
for each neighbor v:
    DFS(v)
```

👉 The key is: `print(u)` **happens FIRST**

✅ That means:

This is **Preorder DFS**

🔄 Compare the three traversals

For a binary tree:

1. Preorder (this question)

Root → Left → Right

→ Matches your code (print first)

- **print in middle → Inorder**
- **print after recursion → Postorder**

So your instinct was close—but inorder would have been:

1, 11, 42, 48, 72, 78, 95 (which is sorted for BST)

👉 That's the giveaway: inorder of a BST = sorted order.

List pseudocode for pre/in/post order DFS in this style

Here's all three traversals written in the **same DFS-style pseudocode** as your question:

🌳 Preorder DFS (Root → Left → Right)

```
DFS(u):
    print(u)
    for each v adjacent to u (left child first, then right child):
        if not visited[v]:
            visited[v] = true
            DFS(v)
```

👉 **Print happens first**

🌳 Inorder DFS (Left → Root → Right)

```
DFS(u):
    mark u as visited

    if left child exists and not visited:
        DFS(left child)

    print(u)

    if right child exists and not visited:
        DFS(right child)
```

👉 **Print happens in the middle**

---

2. Inorder

Left → Root → Right

→ Root is printed in the middle ❌ (not this)

---

3. Postorder

Left → Right → Root

→ Root printed last ❌ (previous question)

🔍 Apply to your tree

Tree:

```
        48
      /    \
    11      72
   /  \       \
  1   42      78
                \
                 95
```

Preorder (Root → Left → Right):

1. Start at **48**
2. Go left → **11**
3. Go left → **1**
4. Back → **42**
5. Back to right subtree → **72**
6. Then **78**
7. Then **95**

✅ Final answer:

48, 11, 1, 42, 72, 78, 95

⚡ Quick memory trick

- **print before recursion → Preorder**

🌳 Postorder DFS (Left → Right → Root)

```
DFS(u):
    mark u as visited

    for each v adjacent to u (left child first, then right child):
        if not visited[v]:
            DFS(v)

    print(u)
```

👉 **Print happens last**

⚡ One-line memory trick

- **Preorder** → print **before**
- **Inorder** → print **between**
- **Postorder** → print **after**

💡 Important note

- The **inorder version only makes sense for trees** (not general graphs), because graphs don't have a clear "left" and "right".
- Preorder and postorder work for **both graphs and trees**

If you want, I can show how to instantly recognize which traversal it is just by glancing at code (very useful in exams).

How to 'Draw a simple directed graph with x strongly connected components using y vertices and z directed

edges. Your graph must be drawn such that by replacing all of its directed edges with undirected edges, a connected undirected graph is produced.'

A clean way to do it is to build the graph out of **blocks**, where each block is one strongly connected component, and then connect the blocks in a **one-way chain** so the whole underlying undirected graph stays connected.

Construction idea

Suppose you want:

- x strongly connected components
- y vertices
- z directed edges

Write the y vertices as x groups:

$$V_1, V_2, \dots, V_x$$

where each group will become one SCC.

Let the sizes be

$$|V_1| = n_1,\ |V_2| = n_2,\ \dots,\ |V_x| = n_x$$

with

$$n_1 + n_2 + \cdots + n_x = y, \qquad n_i \geq 1.$$

Then do two things.

1. Make each group internally strongly connected

For each group V_i, add directed edges so every vertex in that group can reach every other vertex in that same group.

The simplest choice is a directed cycle inside each group:

- if $V_i = \{v_1, \dots, v_{n_i}\}$, add
$$v_1 \to v_2,\ v_2 \to v_3,\ \dots,\ v_{n_i} \to v_1$$
- if $n_i = 1$, add no internal edge

That makes each V_i one SCC.

2. Connect the SCCs in one direction

Add one edge from each block to the next:

$$V_1 \to V_2, \quad V_2 \to V_3, \quad \dots, \quad V_{x-1} \to V_x$$

meaning: choose one vertex in V_i and one in V_{i+1}, and add a directed edge between them.

This keeps the SCCs separate, because edges only go forward, not backward.

Also, if you ignore directions, the graph becomes connected.

Edge count

A directed cycle on $n_i \geq 2$ vertices uses n_i edges, and a 1-vertex SCC uses 0.

So the minimum number of edges in this construction is:

$$\sum_{i:n_i \geq 2} n_i \ + \ (x-1)$$

A very common easy choice is:

- make $x-1$ components single vertices
- put all remaining vertices in one big SCC

Then sizes are:

$$1, 1, \dots, 1,\ y-x+1$$

This gives minimum edges:

$$(y-x+1) + (x-1) = y$$

provided $y - x + 1 \geq 2$. So a very handy fact is:

Easy standard construction:

If $z = y$, you can always do:

- one directed cycle on $y - x + 1$ vertices
- $x - 1$ isolated single-vertex SCCs
- connect them by $x - 1$ one-way edges in a chain

This gives exactly:

- y vertices
- y edges
- x SCCs
- connected underlying undirected graph

How to get more than the minimum number of edges

If you need a larger z, add extra directed edges **inside** components or add extra forward edges between earlier components and later components, as long as you do not create a path back.

Safe options:

- add more edges inside an SCC
- add edges from V_i to V_j with $i < j$

Do **not** add backward edges $V_j \to V_i$ for $j > i$, or two SCCs may merge into one larger SCC.

Existence conditions

To have exactly x SCCs on y vertices, you need at least:

$$1 \leq x \leq y$$

For the graph to be connected after forgetting directions, you need at least:

$$z \geq y - 1$$

because a connected undirected graph on y vertices needs at least $y - 1$ edges.

Also, in a simple directed graph on y vertices, the maximum possible number of directed edges is:

$$y(y-1)$$

if no self-loops are allowed.

So at minimum you need:

$$y - 1 \leq z \leq y(y-1)$$

plus enough structure to keep exactly x SCCs.

Small example

Draw a graph with:

- $x = 3$ SCCs
- $y = 6$ vertices
- $z = 6$ edges

Take components:

- $V_1 = \{1, 2, 3, 4\}$
- $V_2 = \{5\}$
- $V_3 = \{6\}$

Inside V_1, make a directed cycle:

$$1 \to 2,\ 2 \to 3,\ 3 \to 4,\ 4 \to 1$$

Then connect the SCCs in a chain:

$$4 \to 5,\ 5 \to 6$$

Total edges:

$$4 + 1 + 1 = 6$$

SCCs are:

- $\{1, 2, 3, 4\}$
- $\{5\}$
- $\{6\}$

Ignoring directions, the graph is connected.

Template answer

A standard answer is:

1. Partition the y vertices into x groups.
2. In each group, create a directed cycle so that each group is one strongly connected component.
3. Add directed edges from group i to group $i+1$ for $i = 1, \dots, x-1$.
4. Add any remaining edges only within groups or from earlier groups to later groups, so no backward path is created.

That guarantees:

- exactly x SCCs,
- y vertices,
- connected underlying undirected graph.

If you give me specific numbers for x, y, z, I can build one explicitly.

For example what about x,y,z=4,8,10

Yes — $(x, y, z) = (4, 8, 10)$ is doable.

You want:

- **4** strongly connected components
- **8** vertices
- **10** directed edges
- and if directions are ignored, the graph is **connected**

A simple construction is this:

Step 1: Split the 8 vertices into 4 SCCs

Use these groups:

- $C_1 = \{1, 2, 3, 4, 5\}$
- $C_2 = \{6\}$
- $C_3 = \{7\}$
- $C_4 = \{8\}$

That gives 4 components total.

Step 2: Make C_1 strongly connected

Put a directed cycle on the 5 vertices:

$$1 \to 2, \quad 2 \to 3, \quad 3 \to 4, \quad 4 \to 5, \quad 5 \to 1$$

Now $\{1, 2, 3, 4, 5\}$ is one SCC.

So far: **5 edges**

Step 3: Connect the components in a one-way chain

Add:

$$5 \to 6, \quad 6 \to 7, \quad 7 \to 8$$

Now the underlying undirected graph is connected, and the SCCs are still separate.

So far: **8 edges**

Step 4: Add 2 more edges without merging SCCs

Add only **forward** edges, for example:

$$2 \to 7, \quad 3 \to 8$$

Now total edges = **10**.

Final edge set

$$\{(1, 2), (2, 3), (3, 4), (4, 5), (5, 1), (5, 6), (6, 7), (7, 8), (2, 7), (3, 8)\}$$

Why this works

Strongly connected components

- $\{1, 2, 3, 4, 5\}$ is strongly connected because it is a directed cycle.
- $\{6\}$, $\{7\}$, $\{8\}$ are each single-vertex SCCs.
- There are no backward edges, so these SCCs do not merge.

Connected when directions are ignored

If you replace every directed edge by an undirected one, you get connections

$$1 - 2 - 3 - 4 - 5 - 6 - 7 - 8$$

so the graph is connected.

Picture idea

Draw a 5-cycle on the left:

$$1 \to 2 \to 3 \to 4 \to 5 \to 1$$

Then a chain to the right:

$$5 \to 6 \to 7 \to 8$$

Then add two extra forward arrows:

$$2 \to 7, \quad 3 \to 8$$

If you want, I can draw this as a neat ASCII diagram.